

```

#pragma once
#include <random>
#include <string>
#include <vector>
#include "board.h"

using namespace std;

//Information on player/ player node
struct Player{
    string name;
    int money = 1300;
    Property* position = nullptr;
};

class Game{
//Setting up players and roll rng
private:
    Board board_;
    vector<Player> players_;
    int current_ = 0;
    bool rolled_ = false;
    mt19937 rng_{std::random_device{}}();

public:
    Game(){
        //Square places 19
        const char* names[] = {"Go", "Wall St", "Fleet St", "Broadway", "Champs-Elysees",
                                "Oxford St", "Abbey Rd", "Hollywood Blvd", "Fifth Ave", "Ginza St",
                                "Orchard Rd",
                                "Tocayo Ave", "Nanjing Rd", "Las Ramblas", "Nevsky Prospekt", "Via
                                Appia",
                                "Beale St", "Khasoan Rd", "Akihabara"};
        //Cost associated to square places
        const int costs[] = {0, 40, 100, 125, 150, 200, 210, 250, 300, 325, 345,
                              375, 390, 400, 410, 490, 520, 550};

        //Appends both names and cost to the board
        for(int i = 0; i < 16; i++){
            board_.append(names[i], costs[i]);
        }

        //Set the players at start
        for (const char* n : {"Player 1", "Player 2"}){
            Player p;
            p.name = n;
            p.position = board_.start();
            players_.push_back(p);
        }
    }
    //This is just setting up standard functions so that I can call further down below
    const Board& board() const{
        return board_;
    }

    const vector<Player>& players() const{
        return players_;
    }

    int currentIndex() const{
        return current_;
    }

    const Player& currentPlayer() const{

```

```

        return players_[current_];
    }

    bool hasRolled() const{
        return rolled_;
    }
    //////////////////////////////////////

    //Rolling the dice
    string roll(){
        uniform_int_distribution<int> die(1,6);
        int d1 = die(rng_);
        int d2 = die(rng_);

        Player& p = players_[current_];

        bool passed = false;
        p.position = board_.move(p.position, d1 + d2, &passed);
        rolled_ = true;

        string log = p.name + " rolled " + to_string(d1) + " + " + to_string(d2) +
            " and landed on " + p.position->name + ".";

        if (passed){
            p.money += 100;
            log += "\n Passed Go: +$100.";
        }

        Property* square = p.position;

        if(square->cost > 0 && square-> owner != -1 && square->owner != current_){
            int rent = square->cost / 2;
            p.money -= rent;
            players_[square->owner].money += rent;

            log += "\n Owner by " + players_[square->owner].name + ". Paid rent $" + to_string(rent)
+ ".";
        }
        else if(square->owner == current_){
            log += "\n You already own this.";
        }
        return log;
    }

    //Sets up the backend logic for buying sqaures
    bool canBuy() const {
        const Player& p = players_[current_];
        return rolled_ && p.position->cost > 0 && p.position->owner == -1 && p.money >= p.position-
>cost;
    }
    //Sets up the button to buy land (uses canBuy() to preverify buy() option)
    string buy(){
        if(!canBuy()){
            return "Can't buy this property.";
        }
        Player& p = players_[current_];
        p.money -= p.position->cost;
        p.position->owner = current_;
        return p.name + " bought " + p.position->name + " for $" + to_string(p.position->cost) + ".";
    }
    //Sets up the button logic to end turn
    void endTurn(){
        rolled_ = false;
        current_ = (current_ + 1) % static_cast<int>(players_.size());
    }

```

```
};
```